

Generating random topologies

November 28, 2016

0.1 Overview

Given $n \in \mathbb{N}$ and $r \in [0, 1]$, we can build a graph $G_{n,r}$ following the procedure below:

Listing 1: "Procedure to generate a random geometric graph"

```
input: n, r
output: random geometric graph

begin
  nodes = []
  for i in 1..n
    x = uniform(0, 1)
    y = uniform(0, 1)
    n = node(x,y)
    nodes.add(n)

  edges = []
  for n in nodes
    for m in nodes
      if (n != m && distance(n,m) <= r)
        e = edge(n, m)
        edges.add(e)

  return Graph(nodes, edges)
end
```

This kind of graph is known as a **Random Geometric Graph** ($RGG_{n,r}$) with uniform distribution of the nodes over the area $[0, 1]^2$. A RGG has the following property: the expected degree of the vertexes in the graph is related to n and r .

$$Expected-Degree(v) = n\pi r^2$$

Notice that the degree of a vertex is what we call **density**. Hence, if we want to generate a graph with a given density D we must answer the following question:

What values of n and r satisfy the equation $D = n\pi r^2$?

To answer this we simply fix one of them. In particular, we are fixing the number of nodes n to 200. Then it is just a matter of finding the right value for r . Finally, we can use such values to generate the graph by executing procedure 1.

0.2 Computing the map size

In our experimental setup we use three parameters: the number of nodes (n), the density (d) and the transmission range (tx). Observe that we can compute r using n and d . Thus, the transmission range is a way to express r using a different scale. In other words, since the transmission range is not in the interval $[0, 1]$ we must compute the map size. To do that, we first compute the value of r and then we compute the map size using the following equation:

$$m = \lceil \frac{tx}{r} \rceil$$

In this way, we get a squared map of dimensions (m, m) . We can generate a graph using procedure 1 and then scale all the vertexes by m as shown below.

Listing 2: "Procedure to scale a RGG"

```
input: scale, graph
output: random geometric graph

begin
  for n in graph.nodes
    n.x = n.x * scale
    n.y = n.y * scale

  return graph
end
```

0.3 Filtering topologies

After a graph is generated, we are only interested on the positions of its nodes. However, given the random nature of the procedure, we must be sure that the topology has two properties:

1. It is a **connected topology**
2. The average degree is *really* close to the theoretical expected-degree/density (d)

Checking the first property is a classic problem in graph theory. The second property is checked by computing the average degree (*avg*) and testing

the condition $|avg - d| \leq \frac{d \times allowed_error}{100}$ where *allowed_error* is a threshold we use to facilitate the generation of topologies because obtaining the exact degree can be time consuming. In our configuration we are using 10% as *allowed_error*.